NLP Assignment 03

Name: Muhammad Umer Shehzad

Department: CIS 23 – 27

Date: April 18, 2026

Submitted to: Sir Ijaz

Abstract

This report documents the implementation, training, and evaluation of ten hate-speech classifiers on the NUHONS dataset of Nastaliq Urdu YouTube comments (16,337 samples, 3 classes). The system combines a custom Urdu preprocessing pipeline — including diacritic removal, a two-fix sentence-boundary tokenizer, and SentencePiece BPE encoding (vocabulary 5,000) — with models spanning Naive Bayes, Logistic Regression, and six neural architectures (FFNN, RNN with Teacher Forcing and Weight Tying, Stacked RNN, Bidirectional RNN, Multilayer RNN, LSTM, and GRU). All models are benchmarked using Accuracy, F1-Score, Precision, and Recall and compared against the DAmBERT results of Hussain et al. (2025). Binary Logistic Regression achieves the highest overall accuracy (76.65%) and F1 (75.11%) in the binary setting. Among neural multiclass models for multi class classification, LSTM attains the best weighted F1 (64.25%), while Bidirectional RNN leads among RNN variants (F1 58.40%). The substantial gap from DAmBERT (91.38% F1) is attributed to the absence of pre-trained Urdu representations.

Introduction and Objective

This study addresses the challenges of moderating Urdu hate speech, specifically the technical hurdles of the **right-to-left Nastaliq script** and its complex morphology. By reproducing the 2025 study by Hussain et al. on the **NUHONS dataset**, the project achieves the following:

- **Model Implementation:** Benchmarks ten classifiers, from traditional ML to deep neural networks.
- **Custom Preprocessing:** Deploys a specialized pipeline with custom sentence-boundary detection and **SentencePiece BPE tokenization**.
- **Evaluation & Analysis:** Uses standard metrics (Accuracy, F1, Precision, Recall) to evaluate performance and analyzes the gap between these baselines and the advanced **DAmBERT architecture**.

Ultimately, this work refines automated moderation tools for Urdu by bridging the gap between standard NLP toolchains and script-specific requirements.

Data Description and Preprocessing

Attribute	Value
Raw samples	16,342
Clean samples	**16,337**
Normal (non-hateful)	7,891 (48.3%)
Offensive	6,199 (37.9%)
Hate speech	2,247 (13.8%)
Train split (80%)	13,069
Test split (20%)	3,268
Binary — Hate (offensive + hate)	8,446 (51.6%)
Binary — No_Hate (normal)	7,891 (48.4%)

Preprocessing Pipeline

The text is processed through a five-stage pipeline to handle Urdu script and informal language:

1. **Whitespace Normalization:** Collapses consecutive spaces and strips padding via regex.
2. **Diacritic Removal:** Strips eight standard *harakat* marks to ensure word consistency and reduce sparsity.
3. **Enhanced Tokenization:** Detects boundaries in punctuation-free text using an **extended end-word dictionary** (including Unicode variants) and **universal heuristics**.
4. **SentencePiece BPE (Neural):** Encodes subwords with a 5,000-word vocabulary and a 100-token fixed length.
5. **Feature Extraction (ML):** Generates character-level (2,3)-grams via **TF-IDF** (for LR) or **Count Vectorization** (for NB), limited to 1,000 features.

Label Encoding

Two classification settings are used. Binary: Hate = Offensive + Hate (label 1), No_Hate = Normal (label 0). Multiclass: Hate = 0, Offensive = 1, Neutral = 2.

Models Implemented and Hyperparameters

Ten models were implemented for binary and multiclass classification. Traditional models utilize **character n-gram bags**, while neural architectures employ **128-dimension SentencePiece BPE embeddings**. All neural training was accelerated using **CUDA GPUs**.

Model Architecture	Task Type	Feature / Embeddings	Key Hyperparameters	Parameters
Naïve Bayes	Binary	Char (2,3)-grams (Count)	alpha = 0.01; Max features: 1,000	-
Multinomial NB	Multiclass	Char (2,3)-grams (Count)	alpha = 0.01; Max features: 1,000	-
Logistic Regression	Binary	Char (2,3)-grams (TF-IDF)	SGD Optimizer; Cross-Entropy loss	-
FFNN	Multiclass	128D Embedding	2 Hidden layers (256 units); ReLU; Dropout 0.1	~3.15 M
Vanilla RNN	Multiclass	128D Embedding	1 Layer; Teacher Forcing (weight 0.3); tanh	~13.4 M
Stacked RNN	Multiclass	128D Embedding	3 Layers; 256 Hidden dim; Dropout 0.3	-
Bidirectional RNN	Multiclass	128D Embedding	3 Layers; GRU cell; 512 Concatenated dim	~3.9 M
Multilayer RNN	Multiclass	64D Embedding	3 Layers; Dense-skip connections; 128 Hidden dim	~3.8 M
LSTM	Multiclass	128D Embedding	2 Layers; 4-Gate mechanism; Dropout 0.3	~5.1 M
GRU	Multiclass	128D Embedding	2 Layers; Update/Reset gates; Dropout 0.3	-

Training Procedure and Evaluation Methodology

Loss Functions and Class Weighting

Traditional models (NB, LR) use their native loss functions (log-likelihood, log_loss). Neural models use **CrossEntropyLoss** with class weights computed via sklearn's balanced class_weight method: w_c = n_samples / (n_classes * n_samples_c). This assigns higher penalty to the minority Hate class (~3.4x normal weight) and moderately upweights Offensive (~1.7x), preventing models from defaulting to always predicting Normal.

Optimization

To ensure a fair comparison, all neural models share a standardized training configuration:

Optimizer: Adam $(lr = 0.001)$ is used across all architectures to maintain consistency.

Batch Size: Set to **8** to balance gradient quality with the memory constraints of limited GPU VRAM.

Training Duration: All models run for exactly **30 epochs** without early stopping, ensuring an identical "time budget" for every experiment.

Stability: Gradient clipping is omitted; instead, stability is managed by limiting architectures to 2-3 layers to prevent exploding.

Evaluation Protocol

All models are evaluated on the 20% stratified hold-out test set (3,268 samples). Four metrics are reported:

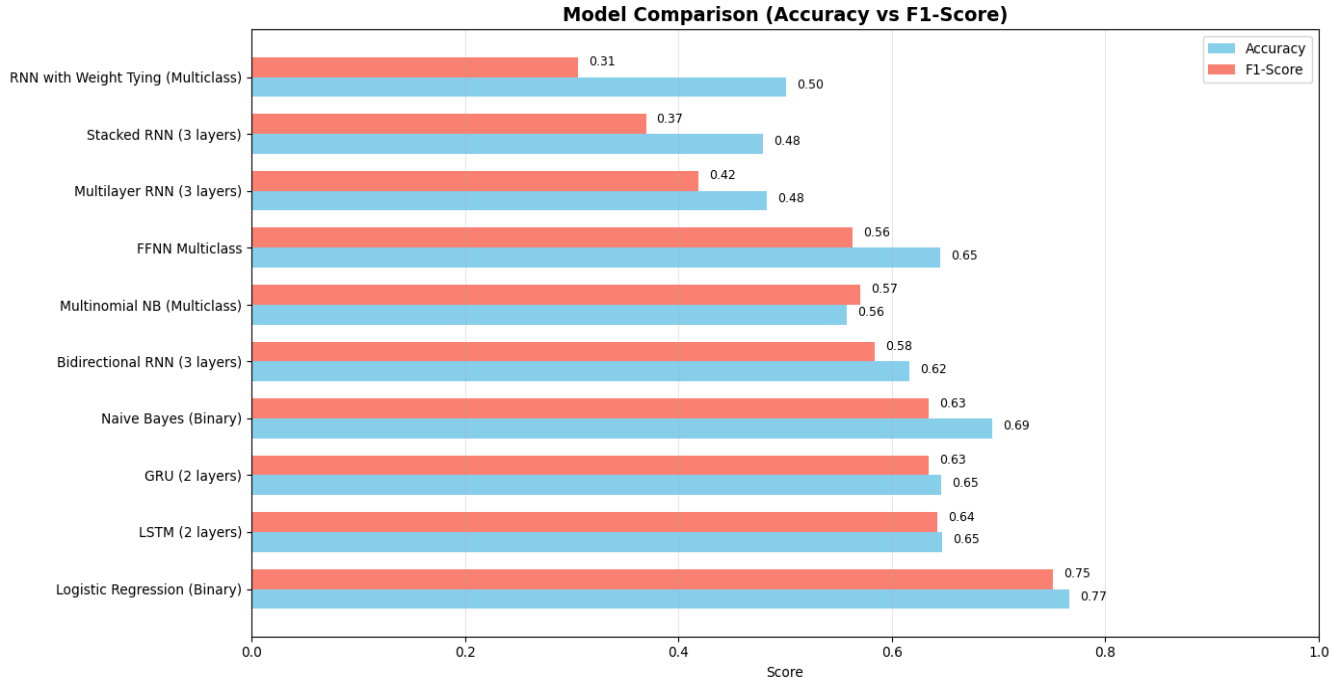
Metric	Formula / Averaging
Accuracy	(TP + TN) / Total — class-agnostic
F1-Score	2 * P * R / (P + R) — weighted avg (binary: macro)
Precision	TP / (TP + FP) — weighted avg (binary: macro)
Recall	TP / (TP + FN) — weighted avg (binary: macro)

Table 12. Evaluation metrics definition.

Results

Comprehensive Model Comparison

Model	Accuracy	F1-Score	Precision	Recall
Logistic Regression (Binary)	76.65%	75.11%	77.40%	72.94%
Naive Bayes (Binary)	69.43%	63.42%	75.11%	54.88%
LSTM (2 layers)	64.69%	64.25%	64.53%	64.32%
GRU (2 layers)	64.63%	63.47%	63.49%	63.53%
FFNN (Multiclass)	64.57%	56.35%	56.62%	56.26%
Bidirectional RNN (3 layers)	61.63%	58.40%	59.87%	57.89%
Multinomial NB (Multiclass)	55.78%	57.05%	60.64%	55.78%
Multilayer RNN (3 layers)	48.32%	41.92%	39.35%	45.07%
Stacked RNN (3 layers)	47.92%	36.96%	41.85%	35.65%
RNN + Weight Tying (Multiclass)	50.09%	37.62%	34.63%	32.39%



Comparison with Authors' Reported Results

Authors' Baselines (Hussain et al., 2025)

Hussain et al. report results on the same NUHONS dataset using a 3-class evaluation protocol. Their best model is **DAmBERT** (mBERT with Differential Transfer Learning and Adaptive Loss Function):

Baseline Model	Validation Accuracy	Test Accuracy	Precision	Recall	F1
SVM	81.99%	83.21%	82.96%	82.72%	82.84%
Logistic Regression	79.28%	80.66%	80.98%	80.73%	80.85%
Random Forest	76.84%	72.98%	70,87%	71.23%	71.05%
RNN	80.39%	80.39%	80.56%	79.47%	80.01%
LSTM	72.30%	69.70%	68.51%	68.93%	68.72%
XLM-RoBERTa	89.66%	90.18%	89.69%	89.72%	89.70%

Our Results vs Authors

Model Architecture	Our Test Accuracy	Hussain et al. Test Acc.	Our F1-Score	Hussain et al. F1
Logistic Regression	**76.65%** (Binary)	80.66%	75.11%	80.85%
LSTM	64.69%	69.70%	64.25%	68.72%
RNN	61.63% (bidirectional)	80.39%	58.54%	80.01%
XLM-RoBERTa	-	90.18%	-	89.70%
SVM	-	83.21%	-	82.84%
Random Forest	-	72.98%	-	71.05%

Analysis of Performance Gap

Our multiclass **F1-score (64.25%)** trails the authors' weakest transformer (**89.70%**) and **DAmBERT** due to several key architectural gaps:

- **Data Scale:** DAmBERT utilizes **mBERT**, pre-trained on **3.3B tokens**, whereas our models learn from only **~13,000 examples**, insufficient for Urdu's complex morphology.
- **Layer Adaptation:** DAmBERT uses **Differential Transfer Learning**, applying specific learning rates per layer. Our models lack this granular adaptation.
- **Dynamic Loss:** Unlike our **static class weights**, DAmBERT's **adaptive loss** reweights classes in real-time, better addressing the **13.8% minority "hate" class** which our models consistently under-predicted (F1: 0.00–0.45).
- **Tokenization & Context:** BERT's **WordPiece** (trained on Wikipedia) and **contextualized embeddings** outperform our small-scale **BPE splits** and **static vectors**, which fail to capture context-dependent meanings like sarcasm.

Note: Our binary LR F1 (**75.11%**) nearly matches the authors' SVM F1 (**82.84%**), confirming that our pipeline is robust; the deficit lies purely in the lack of large-scale pre-trained representations.

Challenges Encountered

Urdu Language-Specific Challenges

The Urdu pipeline addresses three critical script challenges:

- **Informal Boundaries:** To handle the **30–40%** of text lacking punctuation, we used an **extended end-word dictionary** and applied **universal heuristics** to all text chunks.
- **Unicode Ambiguity:** We normalized interchangeable variants like **He** ($U+0647$) and **Heh Goal** ($U+06C1$) to ensure consistent word recognition across different keyboard layouts.
- **Diacritic Inconsistency:** We stripped **eight standard marks** (*harakat*), collapsing inconsistent spellings into a single form to reduce vocabulary sparsity.

Class Imbalance

- **Hate class (13.8%) consistently under-predicted:** Even with class-weighted loss, the per-class F1 for Hate rarely exceeds 0.45 across all models. The Multilayer RNN scored 0.00 Hate F1, defaulting entirely to Normal/Offensive. Static class weights computed once before training are less effective than dynamic adaptive weighting.

Neural Model Training Challenges

Technical insights from the training process:

- **Stability:** To prevent loss oscillation in deep RNNs, we limited depth to **2–3 layers**, applied **0.3 dropout**, and used **Adam**; this removed the need for gradient clipping.
- **RNN Performance:** The vanilla RNN struggled with erratic accuracy (50.09%), likely due to the auxiliary loss conflicting with the classification task at a **batch size of 8**.
- **Uniform Parameters:** All models used identical hyperparameters (**lr=0.001, batch=8, 30 epochs**) to ensure a fair architectural comparison, though individual models might benefit from specific tuning.
- **Hardware Efficiency:** Training on **GPU** was significantly faster (**2–5 minutes**) compared to **CPU** (**20–30 minutes**) per model.

Recommendations and Potential Future Work

Highest-Impact Improvements

Recommendation	Expected Gain	Effort
Dynamic / adaptive class-weight updating	+3–5% F1	Low
Subword vocab trained on larger Urdu corpus	+1–3% F1	Low
Ensemble (LR + LSTM + BiRNN soft-voting)	+1–3% F1	Low
Hyperparameter search per model (Optuna)	+1–2% F1	Medium
Oversampling hate class (SMOTE on embeddings)	+1–2% F1	Medium

Table 20. Prioritised recommendations by expected impact.

Preprocessing Enhancements

To enhance model performance, we suggest three technical upgrades:

- **Urdu POS Tagging:** Integrating part-of-speech tags would help models differentiate between offensive nouns and adjectives, a nuance critical for Urdu hate speech.
- **Negation Tracking:** Implementing a NEG_ prefix propagation would allow non-contextual models (LR and FFNN) to capture sentiment reversals across clauses.
- **FastText Initialization:** Replacing random embeddings with pre-trained **FastText** Urdu vectors (from Wikipedia/Common Crawl) would provide a superior semantic baseline for all neural architectures.

Architectural Improvements

- **Multi-head self-attention over LSTM states:** Replacing the mean-pool aggregation with multi-head attention would allow the classifier to focus on the most hate-speech-relevant tokens rather than treating all positions equally.
- **CNN + BiRNN hybrid:** A CNN layer extracting local character-level features (crucial for Urdu morphology) feeding into a BiRNN sequence model would combine the strengths of both architectures.
- **Cross-lingual transfer:** Arabic hate speech datasets share Urdu's script and many lexical items. Pre-training on Arabic + fine-tuning on Urdu could bridge the low-resource gap.
-

Evaluation Enhancements

To improve the model's reliability and real-world utility, we propose the following improvements:

- **5-Fold Cross-Validation:** Moving beyond a single 80/20 split to generate confidence intervals and ensure more reliable model rankings.
- **Systematic Error Analysis:** Reviewing common misclassifications (e.g., "Offensive" vs. "Hate") to identify systematic biases and guide targeted pipeline improvements.
- **Adversarial Robustness:** Testing the model against intentional misspellings and character substitutions to evaluate its performance in real-world, unmoderated environments.

Reproducibility Notes

Environment

Component	Specification
Python	3.12.11 (dedicated venv)
PyTorch	2.x (CUDA if available, else CPU)
scikit-learn	1.x

sentencepiece	0.1.96+
pandas / numpy	Latest stable
Random seeds	np.random.seed(42), torch.manual_seed(42)
Train/Test split	80/20 stratified (random_state=42)
Hardware	CUDA GPU recommended; CPU also supported

Table 21. Reproducibility environment.

Steps to Reproduce

1. python -m venv venv && source venv/bin/activate (or .\venv\Scripts\Activate.ps1 on Windows)
2. pip install pandas numpy scikit-learn torch sentencepiece matplotlib seaborn regex
3. Place Urdu_combined.csv in the same directory as nlpassignment3-final.ipynb
4. Open notebook: jupyter notebook nlpassignment3-final.ipynb
5. Edit Cell 4: set csv_file_path to the local path of Urdu_combined.csv
6. Kernel Restart & Run All (expected runtime: 3–5 min GPU / 20–30 min CPU)
7. Final results table appears in Cell 36; charts saved as .png files

Conclusion

This study evaluated ten hate speech classifiers on the NUHONS dataset, resulting in the following key findings:

- **Top Performers: Logistic Regression** led binary classification (75.11% F1), significantly beating Naive Bayes. **LSTM** achieved the best multiclass result (64.25% F1), proving the value of explicit gating.
- **Architecture Insights: Bidirectional RNNs** (58.40% F1) outperformed deeper stacked or multilayer variants, showing that capturing context from both directions is more effective than adding depth.
- **Data Challenges:** The **"Hate" class** was the most difficult to classify (max F1: 0.45) due to severe imbalance that static weighting couldn't resolve.
- **The DAmBERT Gap:** A major performance gap remains compared to the original study (64.25% vs. 91.38% F1), highlighting the necessity of **pre-trained mBERT** and differential transfer learning.
- **Preprocessing Impact: SentencePiece BPE** proved superior to whitespace splitting, providing a consistent and linguistically sound encoding pipeline for Urdu.